



Desenvolvimento Web

Profª Sabrina B. Moreira



Aprendendo CSS 5

Aula 13.1: media query e display grid

CSS Media Query

CSS: media query

- ❖ Surgiu no **CSS 3**, como uma evolução dos media types, que já existia na versão anterior.
- ❖ Em **2010**, o World Wide Web Consortium (W3C) introduziu oficialmente os Media Queries no CSS3.
- ❖ Esse avanço foi impulsionado pela **popularização dos smartphone** que exigiu sites adaptáveis a telas menores.

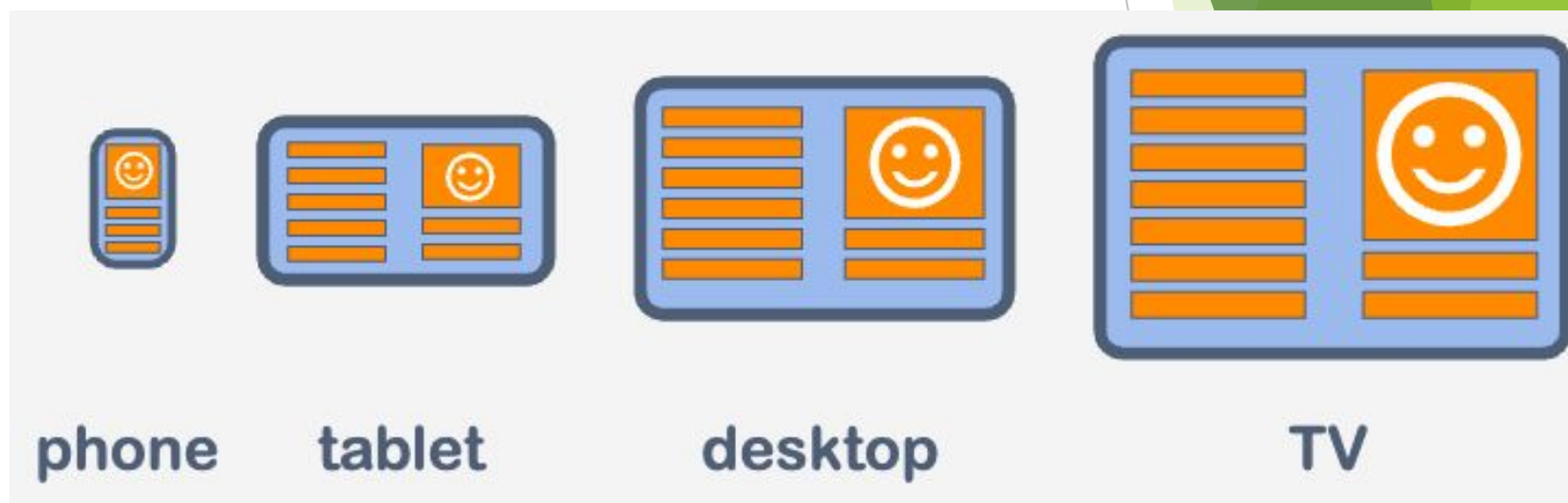


```
body {  
  font: x-small  
  background: #  
  color: black;  
  margin: 0;  
  padding: 0;
```

CSS: media query

O Media Query permite aplicar estilos CSS com base em condições como:

- ❖ largura da tela
- ❖ altura
- ❖ orientação
- ❖ resolução
- ❖ tipo de dispositivo

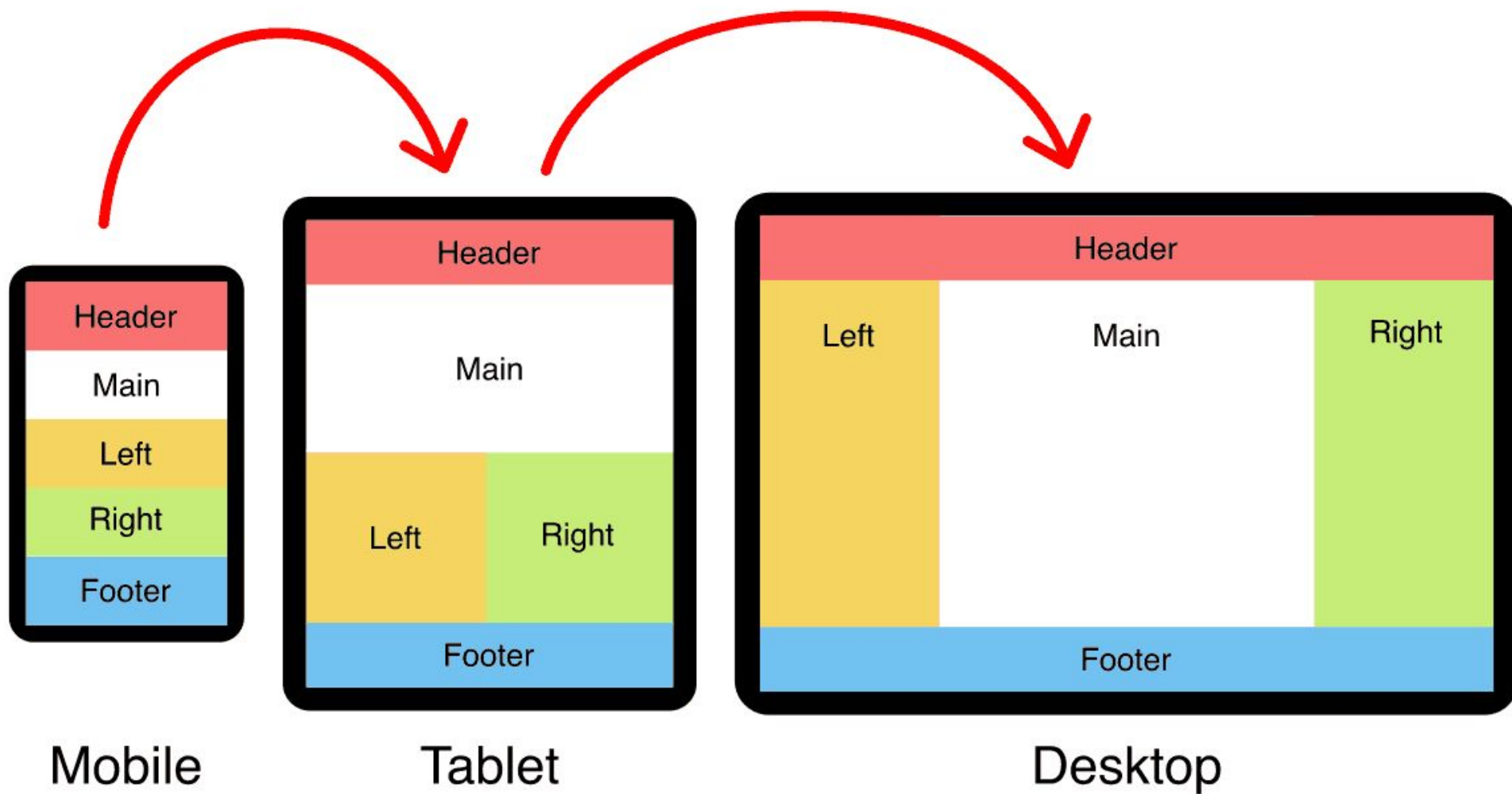


Ele faz o site “responder” ao ambiente onde está sendo exibido

CSS: media query

- ❖ Permitem aplicar estilos diferentes de acordo com as **características do dispositivo**, como largura da tela, altura, orientação (retrato ou paisagem) e até resolução.
- ❖ Ajudam a **adaptar o layout** para funcionar bem em celulares, tablets e computadores.

CSS: media query



CSS: media query

- ❖ Funcionam como condições, ou seja, você **define uma regra** que só será aplicada se determinada condição for verdadeira.
- ❖ Por exemplo, é possível mudar o tamanho de um texto ou reorganizar elementos da página quando a tela for menor que 768px.



CSS: media query

- ❖ Elas são muito utilizadas junto com layouts fluidos e imagens flexíveis, sendo uma das bases do design responsivo.
- ❖ Permitindo que o site se **adapte continuamente** sem precisar criar várias versões separadas.

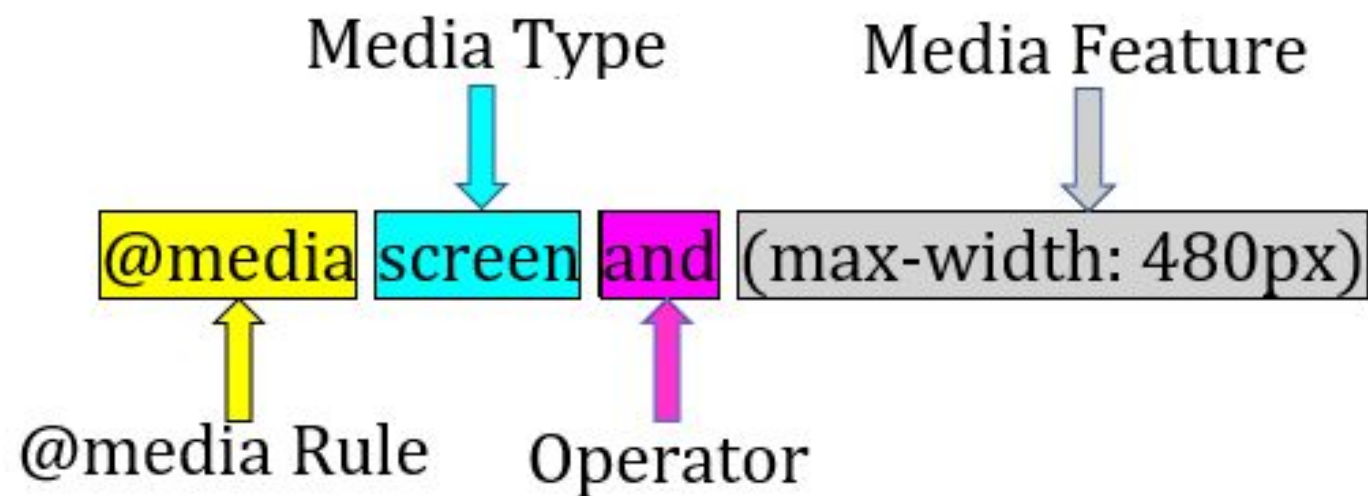
```
@media (condição) {  
    /* regras CSS */  
}
```

Fonte: autora, 2026

CSS: media query: estruturação da regra

É estruturado por:

- ❖ Rule (regra)
- ❖ Type (tipo)
- ❖ Operator (operador)
- ❖ Feature (condição)



Fonte: autora, 2026

CSS: media query: tipo de regras

Por largura:

- ❖ **max-width** (mobile-first invertido):

Aplica estilos para telas menores que 768px.

```
@media (max-width: 768px) {  
  h1 {  
    font-size: 35px;  
  }  
}
```

Fonte: autora, 2026

CSS: media query: tipo de regras

Por largura:

❖ **min-width** (mobile-first invertido):

Aplica estilos para telas maiores que 768px.

```
@media (min-width: 768px) {  
  h1 {  
    font-size: 35px;  
  }  
}
```

Fonte: autora, 2026

CSS: media query: tipo de regras

Por intervalo de largura:

❖ min-width e
max-width:

Aplica estilos para telas
maiores que 768px e
menores que 1024px.

```
@media (min-width: 768px) and (max-width: 1024px) {  
  h1 {  
    font-size: 35px;  
  }  
}
```

Fonte: autora, 2026

Usado para definir estilos
específicos para tablets.

CSS: media query: tipo de regras

Por orientação de tela: verifica a relação entre largura e altura da viewport (tela visível):

- ❖ **portrait** (vertical): altura maior que largura (tela “em pé”), $\text{height} \geq \text{width}$.
- ❖ **landscape** (horizontal): largura maior que altura (tela “deitada”), $\text{width} > \text{height}$.

```
@media (orientation: portrait) {  
  h1 {  
    font-size: 35px;  
  }  
}  
  
@media (orientation: landscape) {  
  h2 {  
    font-size: 22px;  
  }  
}
```

CSS: media query: tipo de regras

Por tipo de mídia:

- ❖ **screen**: aplica as regras em tudo que possui tela: computador, celular, tablet, smart TV...
- ❖ **print**: aplica os estilos no momento da impressão ou exportação de PDF, para deixar o conteúdo legível no papel.

```
@media screen {  
  h1 {  
    font-size: 35px;  
  }  
}  
  
@media print {  
  .header,  
  .footer {  
    display: none;  
  }  
}
```


CSS: principais breakpoints

Aqui temos os principais breakpoints, que vão indicar o momento que o layout vai se ajustar de acordo com o **tipo de regra** aplicado.

Eles vieram historicamente de dispositivos populares:

- ❖ 480px: celulares antigos
- ❖ 768px: tablets como o iPad
- ❖ 1024px: largura comum de tablets maiores e notebooks antigos

Hoje existem muitos tamanhos diferentes, por isso esses valores são só **referência**.

```
/* Mobile maior */
@media (min-width: 481px) {
  body {
    font-size: 15px;
  }
}

/* Tablet */
@media (min-width: 769px) {
  body {
    font-size: 16px;
  }
}

/* Desktop */
@media (min-width: 1025px) {
  body {
    font-size: 18px;
  }
}
```


CSS: principais breakpoints

- ❖ Hoje, a regra indicada é ser mobile-first.
- ❖ Ou seja, o layout padrão é para telas pequenas, e as excessões das regras dos breakpoints serem aplicados a telas grandes.

```
/* Base: mobile pequeno (até 480px) */  
body {  
  font-size: 14px;  
}
```

```
/* Mobile maior */  
@media (min-width: 481px) {  
  body {  
    font-size: 15px;  
  }  
}
```

```
/* Tablet */  
@media (min-width: 769px) {  
  body {  
    font-size: 16px;  
  }  
}
```

```
/* Desktop */  
@media (min-width: 1025px) {  
  body {  
    font-size: 18px;  
  }  
}
```

CSS Grid

CSS: Diferença de flexbox e grid

- ❖ O **CSS Flexbox** cria um layout de uma dimensão, ou seja, linhas ou colunas.
- ❖ Já o **CSS Grid** cria layouts em duas dimensões, assim sendo, usa linhas e colunas.

Float



Flex



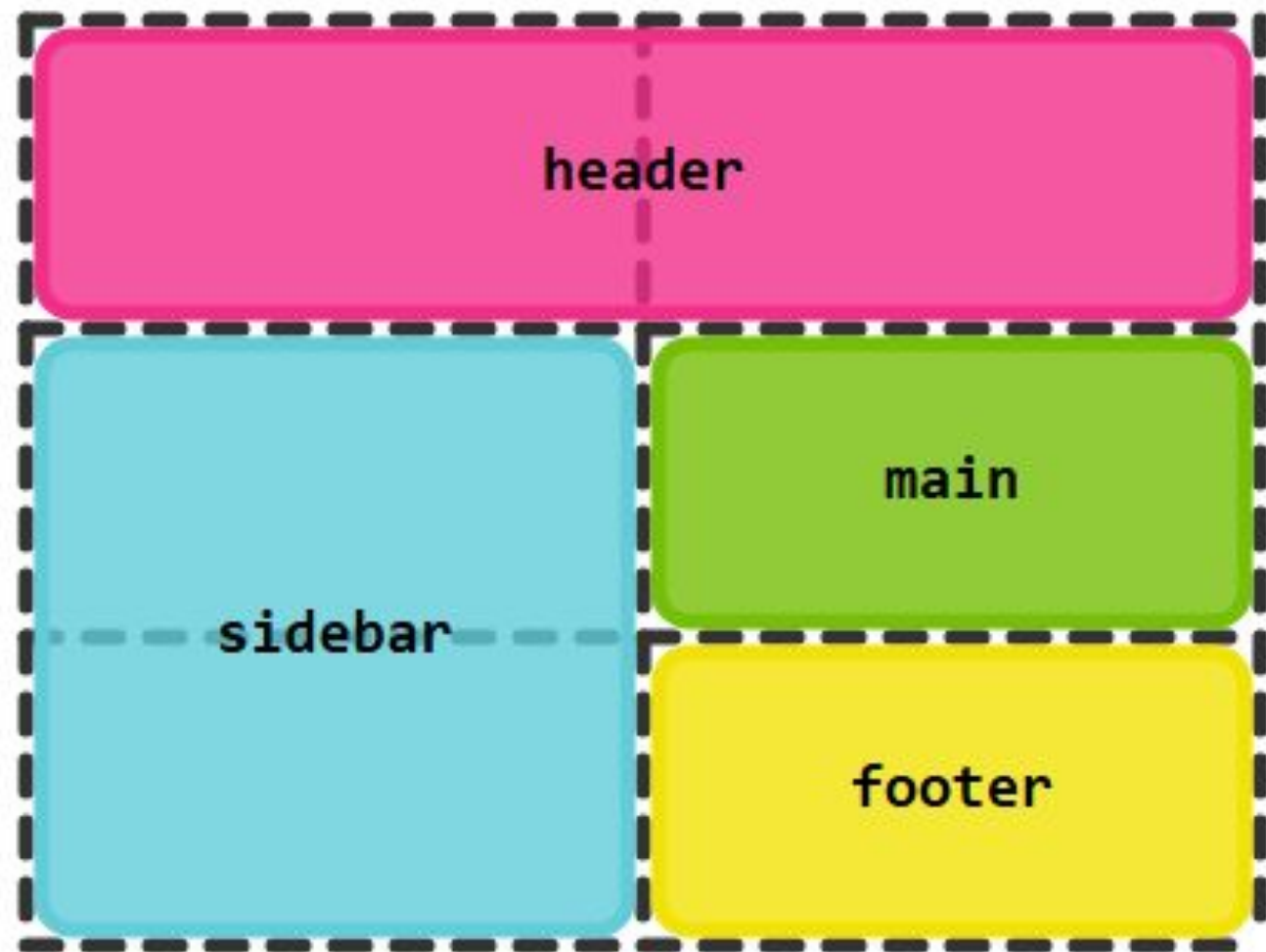
Grid



Fonte: fedev, 2026

CSS Grid: O que é?

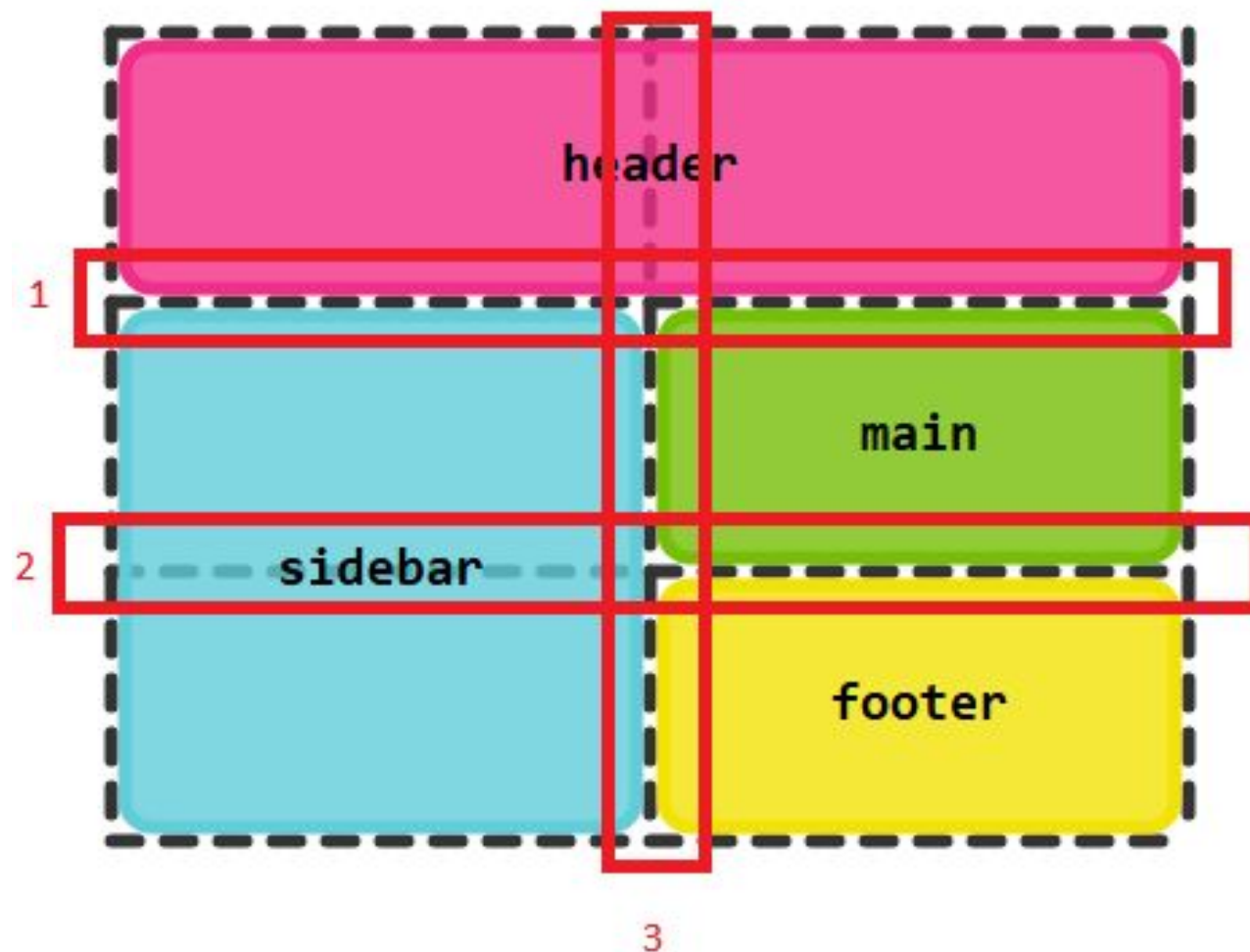
- ❖ O Grid oferece uma forma fácil de fazer design responsivos sem precisar usar floating ou position.



CSS Grid: principais conceitos

❖ Grid Line:

É a linha que define a orientação do layout, se é vertical ou horizontal, sendo enumerado começando com 1.

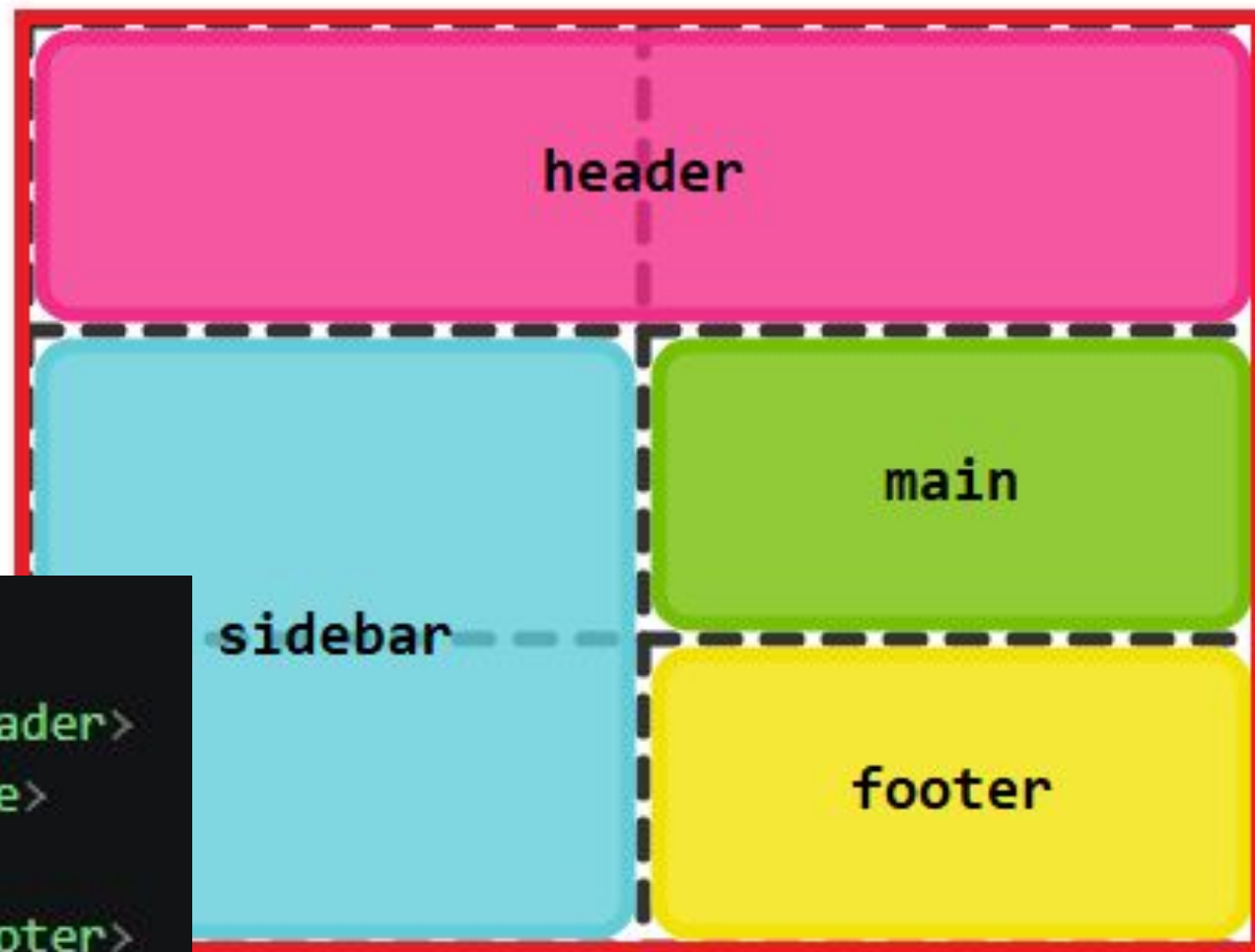


CSS Grid: principais conceitos

❖ Grid container:

O elemento pai que contém a definição de `display grid` ou `inline-grid`.

```
<body class="grid-container">  
  <header class="header grid-item"></header>  
  <aside class="aside grid-item"></aside>  
  <main class="main grid-item"></main>  
  <footer class="footer grid-item"></footer>  
</body>
```



Propriedades para container

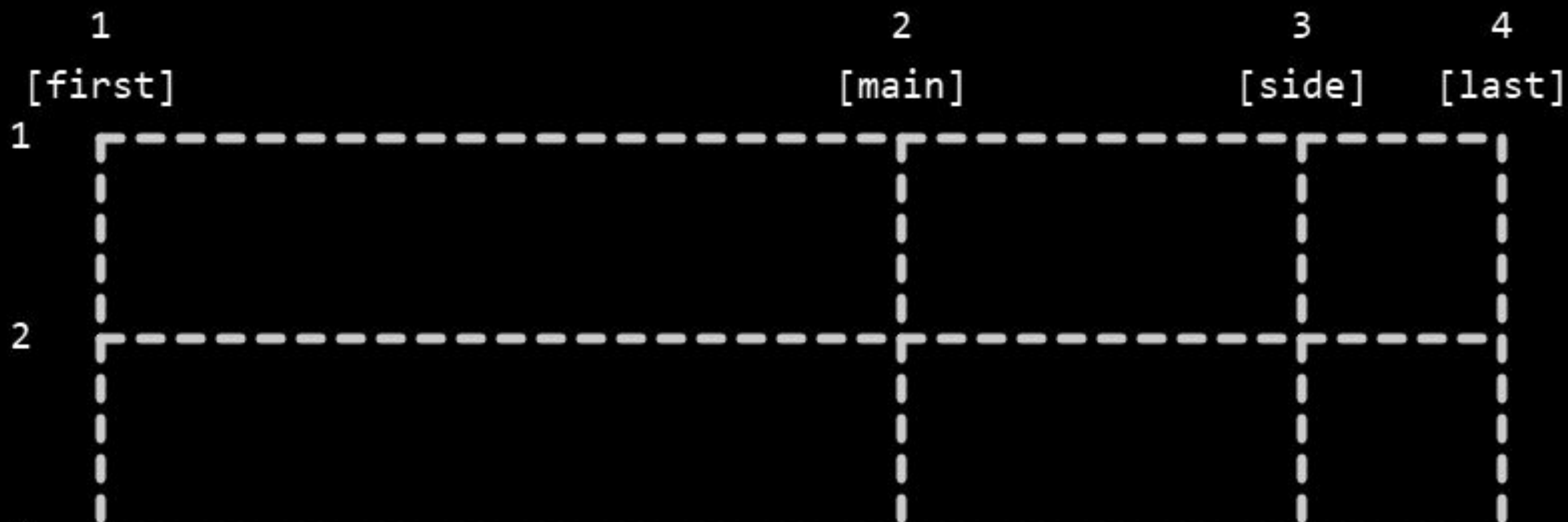
CSS Grid: principais conceitos

❖ grid-template-*:

Definimos as características de linhas e colunas com as propriedades *grid template rows* e *columns*.

```
.grid-container {  
  display: grid; /* grid ou inline-grid */  
  grid-template-rows: 100px 100px; /* 2 linhas */  
  grid-template-columns: 400px 200px 100px; /* 3 colunas */  
}
```


CSS Grid: principais conceitos



Fonte: autora, 2026

CSS Grid: principais conceitos

❖ grid-template-*:

Também se pode ser usado medidas responsivas como porcentagem ou frações.

```
.grid-container {  
  display: grid; /* grid ou inline-grid */  
  grid-template-rows: 50% 50%; /* 2 linhas */  
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas */  
}
```

CSS Grid: principais conceitos

❖ grid:

Ou o equivalente de forma resumida seria como na imagem abaixo.

```
.grid-container {  
  display: grid;  
  grid: 50% 50% / 1fr 1fr 1fr;  
}
```

CSS Grid: principais conceitos

❖ grid-gap:

Já com as propriedades *grid-row-gap* e *grid-column-gap* definimos o espaço entre as linhas e colunas.

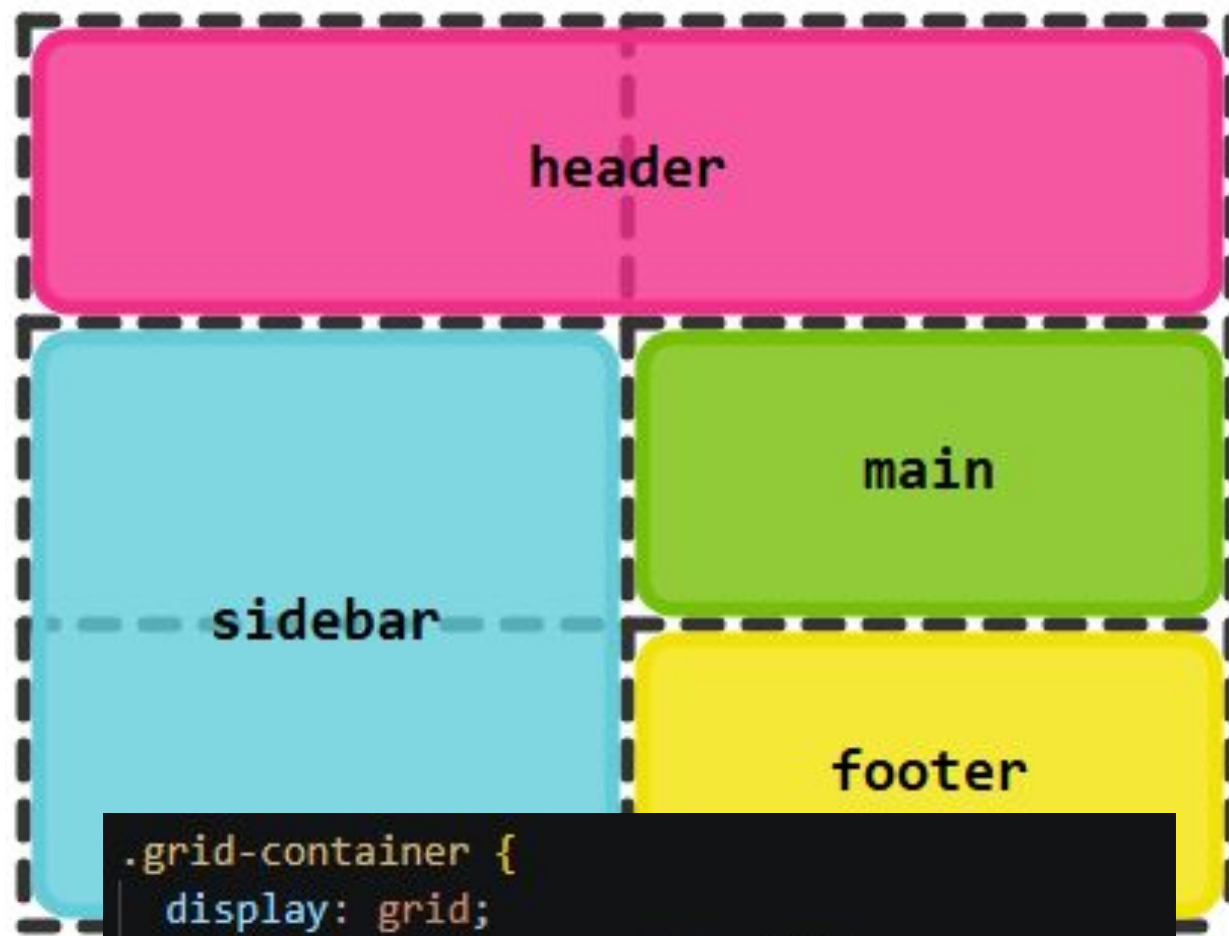
```
.grid {  
  display: grid;  
  grid-template-rows: 100px 100px;  
  grid-template-columns: 400px 200px 100px;  
  grid-row-gap: 10px;  
  grid-column-gap: 20px;  
}
```

CSS Grid: principais conceitos

❖ grid-template-areas:

Define uma área grid referenciado por nomes, onde quanto maior repetição de nomes maior área ocupada.

- ❖ Header ocupa uma coluna e uma coluna vazia (.), aside e main ocupará uma coluna cada e footer duas colunas.



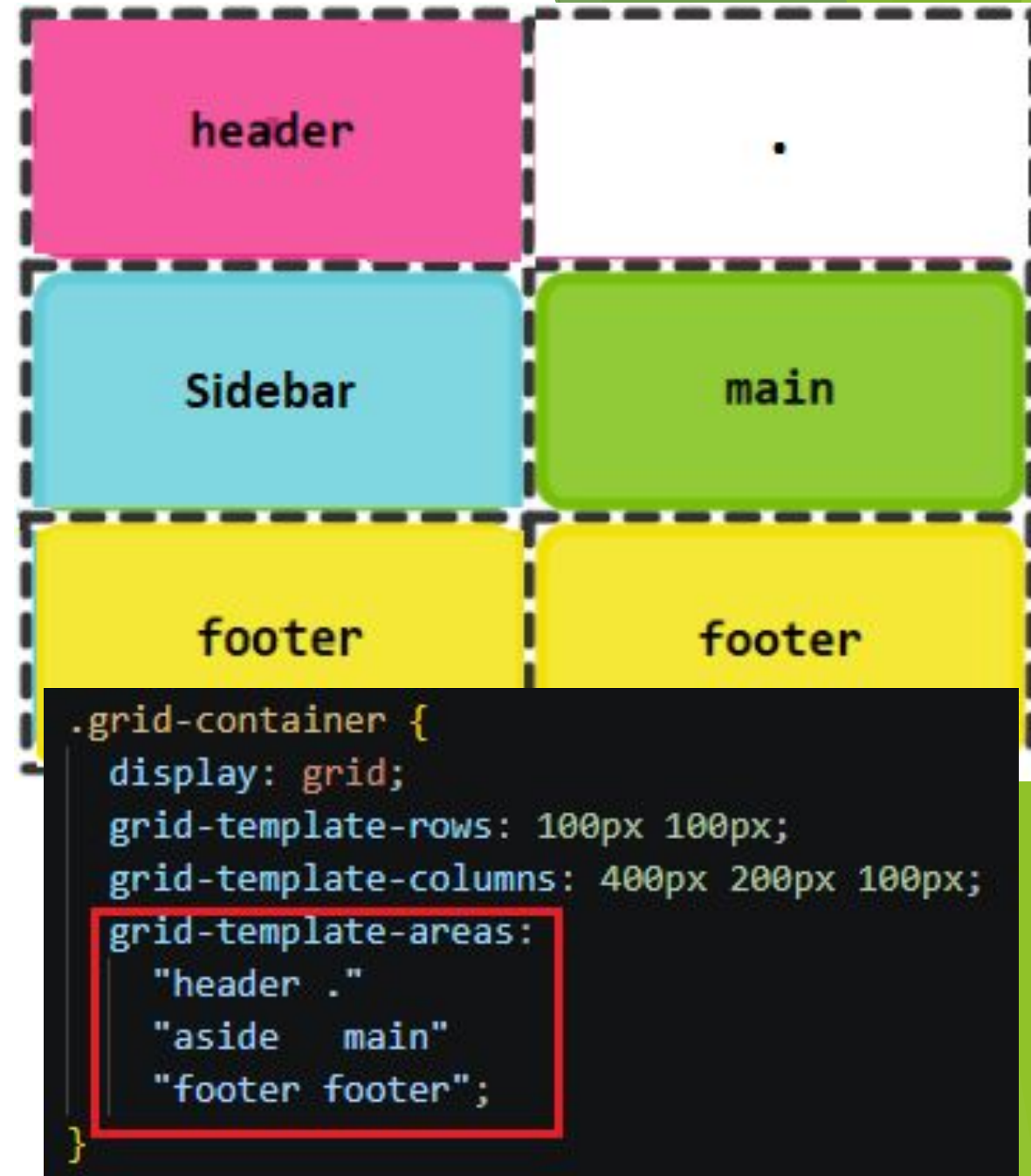
```
.grid-container {  
  display: grid;  
  grid-template-rows: 100px 100px;  
  grid-template-columns: 400px 200px 100px;  
  grid-template-areas:  
    "header header"  
    "aside main"  
    "aside footer";  
}
```

CSS Grid: principais conceitos

❖ grid-template-areas:

Define uma área grid referenciado por nomes, onde quanto maior repetição de nomes maior área ocupada.

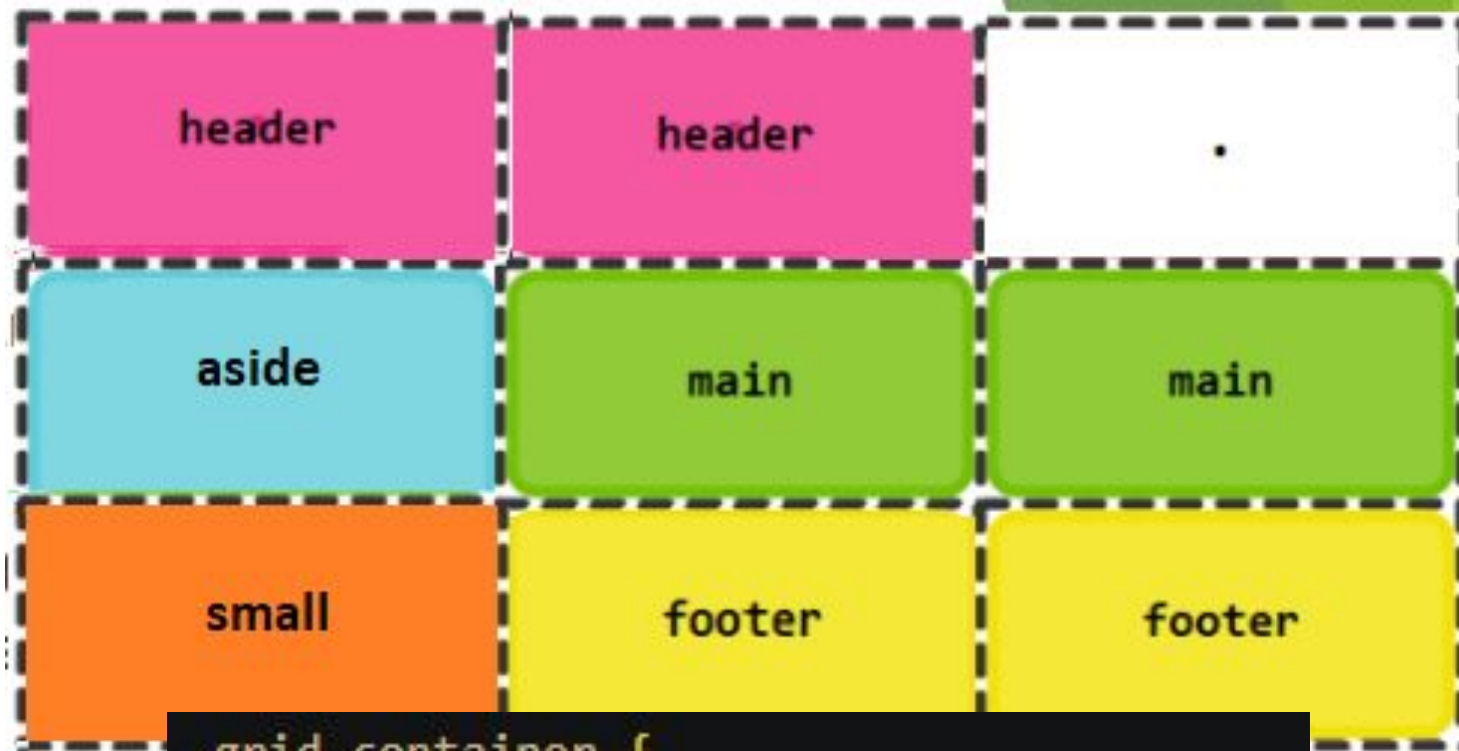
- ❖ Header ocupa uma coluna e uma coluna vazia (.), aside e main ocupará uma coluna cada e footer duas colunas.



CSS Grid: principais conceitos

❖ grid-auto-flow:

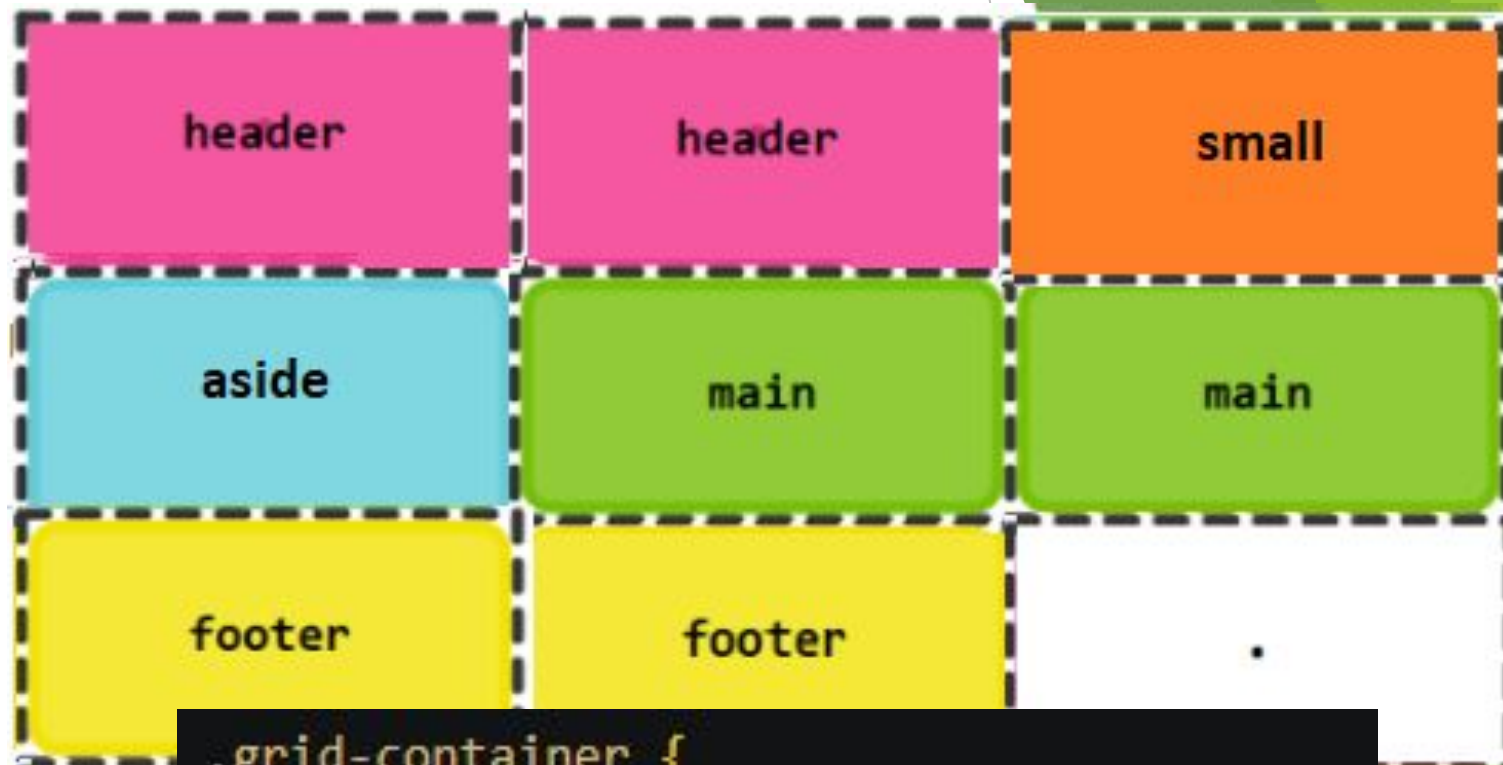
Define como serão organizados os itens na tela quando haver espaços vazios.



```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-auto-rows: 100px;  
  grid-auto-flow: row;  
}
```

CSS Grid: principais conceitos

- ❖ Com dense, evita espaços vazios tenta ordenar elementos.
- ❖ Neste caso, o small que é o último elemento, sobe, e o espaço vazio fica em seu lugar.

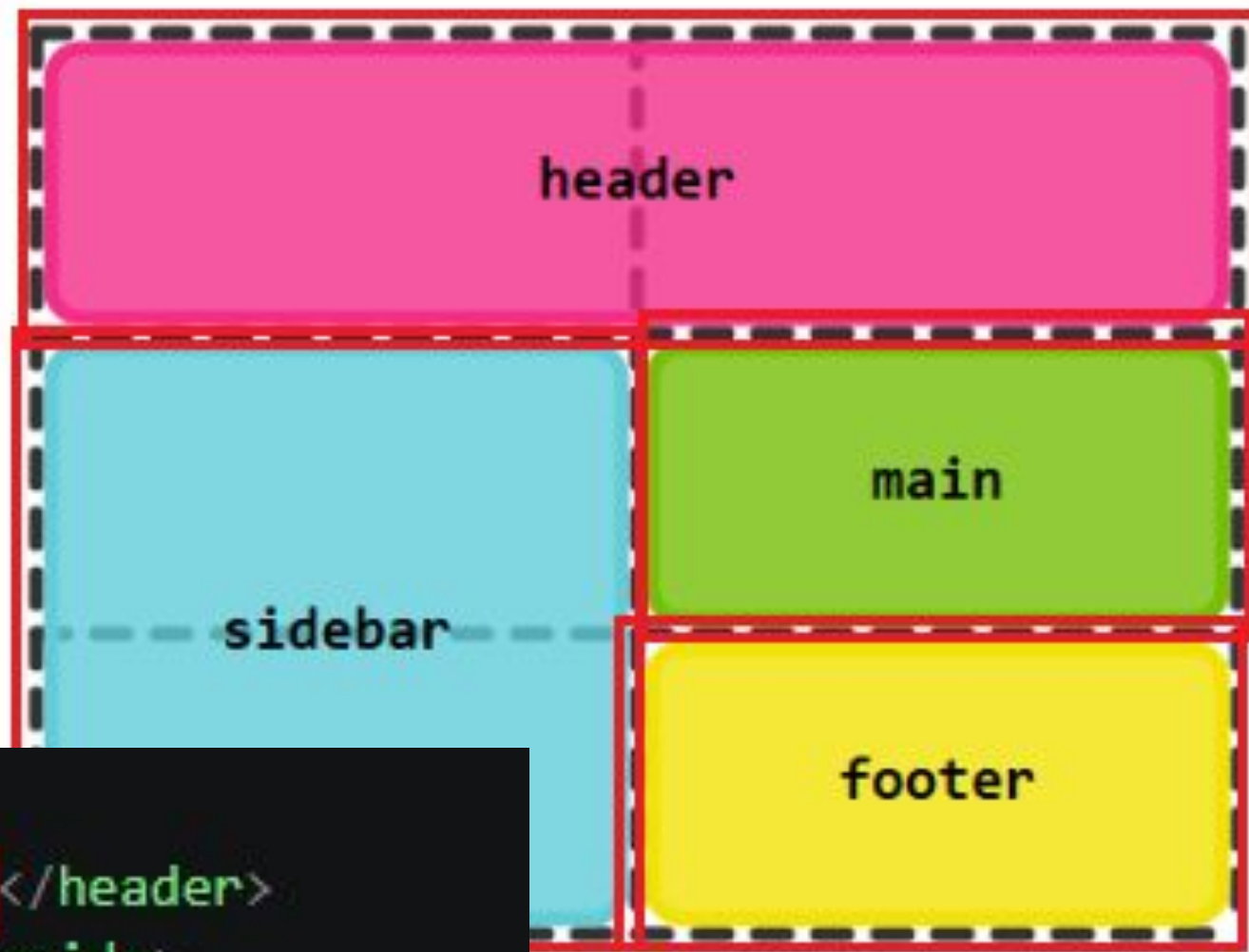


```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-auto-rows: 100px;  
  grid-auto-flow: row dense;  
}
```


CSS Grid: principais conceitos

❖ Grid itens:

Os itens filhos do container grid que serão reorganizados de acordo com as configurações do container.



```
<body class="grid-container">
  <header class="header grid-item"></header>
  <aside class="aside grid-item"></aside>
  <main class="main grid-item"></main>
  <footer class="footer grid-item"></footer>
</body>
```

Propriedades para elementos

CSS Grid: principais conceitos

❖ Grid item:

E para cada **item** filho, é definido o nome do seu *grid-area*.

```
.header.grid-item {  
  grid-area: header;  
}  
  
.aside.grid-item {  
  grid-area: aside;  
}  
  
.main.grid-item {  
  grid-area: main;  
}  
  
.footer.grid-item {  
  grid-area: footer;  
}
```

CSS Grid: principais conceitos

❖ justify-item:

Alinha os itens do grid ao longo do eixo inline (linha, esquerda - direita), ao contrário de align-items, que alinha ao longo do eixo block (coluna, cima - baixo).

- ❖ Esse valor se aplica a todos os itens do grid dentro do container, e controla como os itens se alinham horizontalmente.

```
.grid-container {  
  display: grid;  
  justify-items: center; /* start | end | center | stretch; */  
}
```

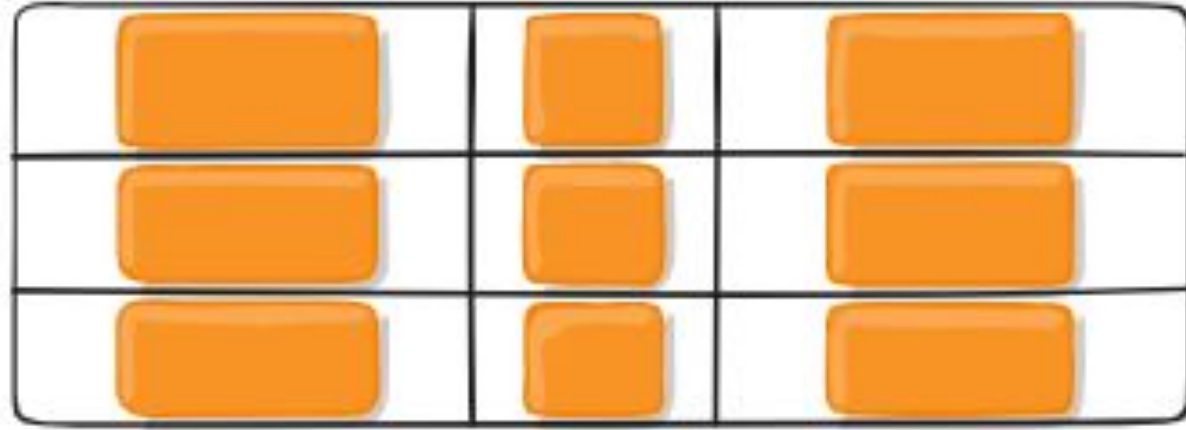
CSS Grid: principais conceitos

❖ justify-item:

```
.grid-container {  
  display: grid;  
  justify-items: start;  
}
```



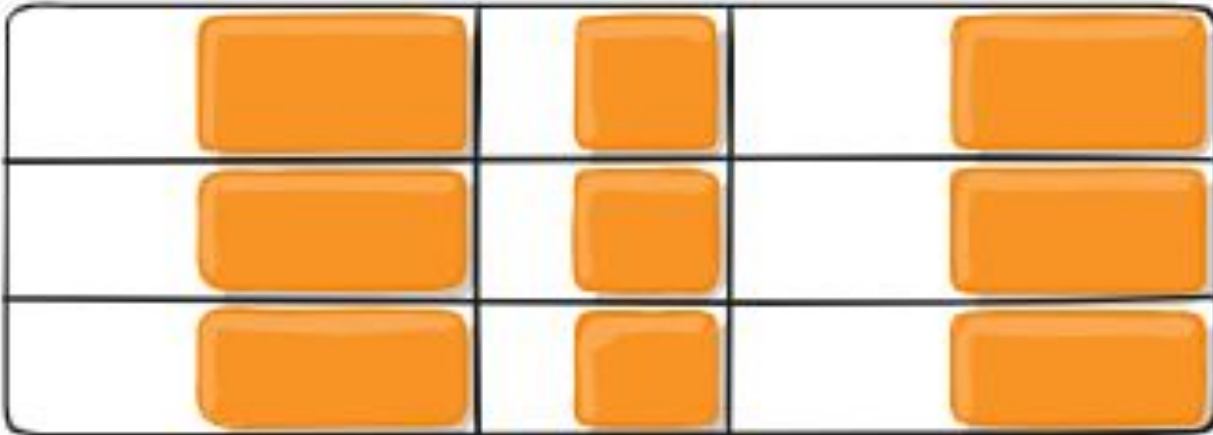
```
.grid-container {  
  display: grid;  
  justify-items: center;  
}
```



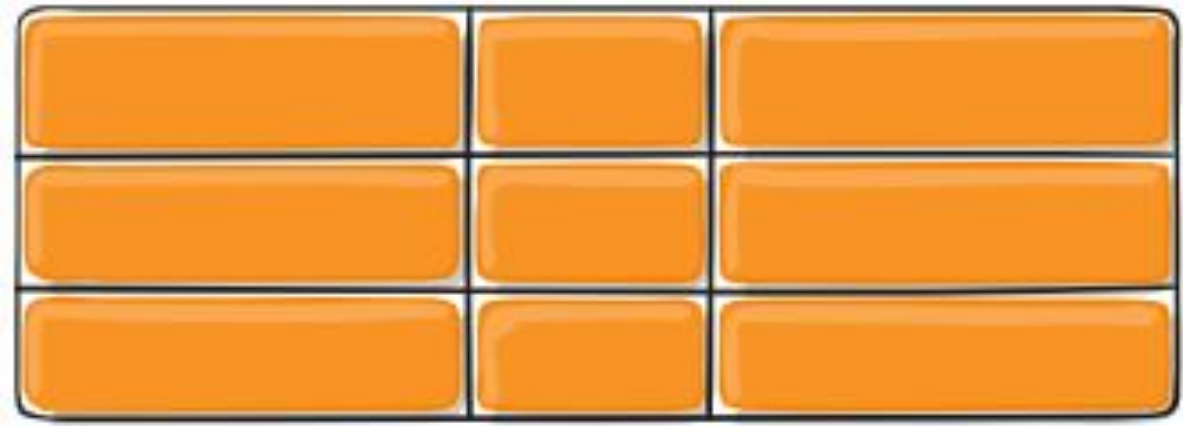
CSS Grid: principais conceitos

❖ justify-item:

```
.grid-container {  
  display: grid;  
  justify-items: end;  
}
```



```
.grid-container {  
  display: grid;  
  justify-items: stretch;  
}
```



CSS Grid: principais conceitos

❖ align-item:

Alinha os itens do grid ao longo do eixo block (coluna, cima/baixo).

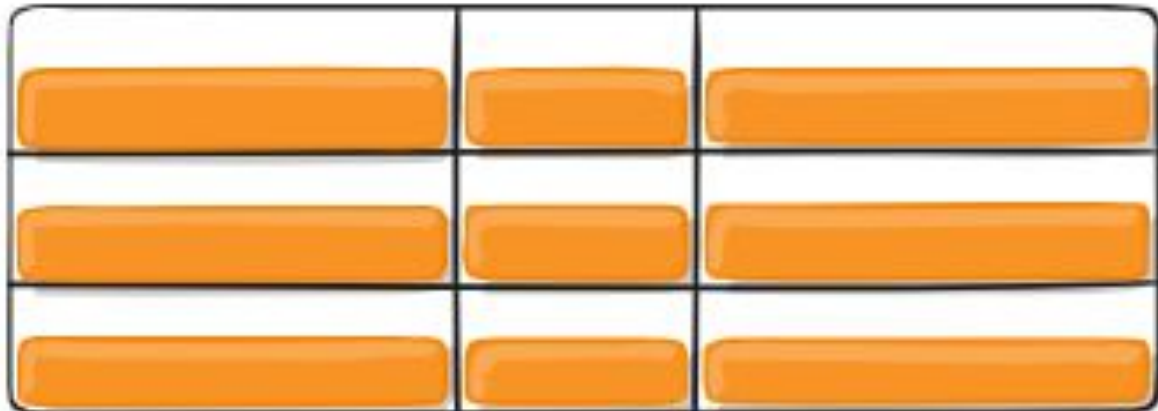
- ❖ Esse valor se aplica a todos os itens do grid dentro do container, e controla como os itens se **alinham verticalmente**.

```
.grid-container {  
  display: grid;  
  align-items: center; /* start | end | center | stretch; */  
}
```

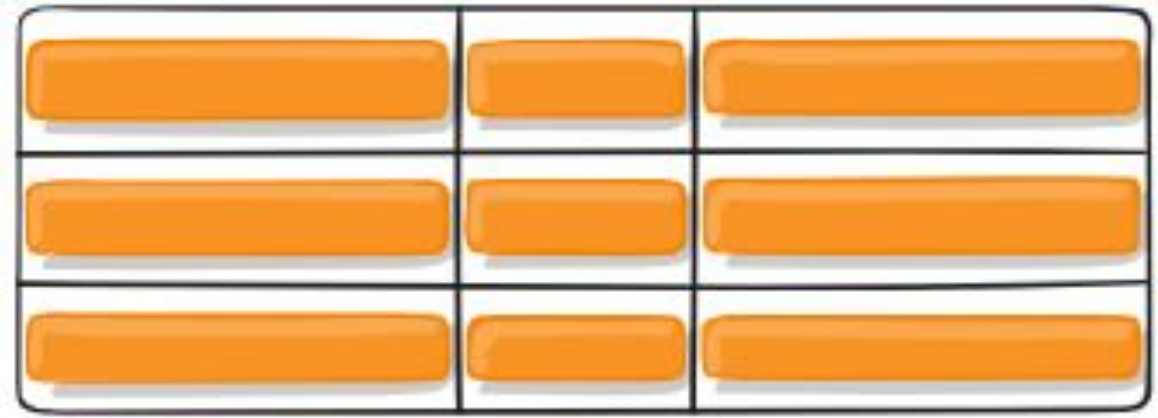

CSS Grid: principais conceitos

❖ align-item:

```
.grid-container {  
  display: grid;  
  align-items: end;  
}
```



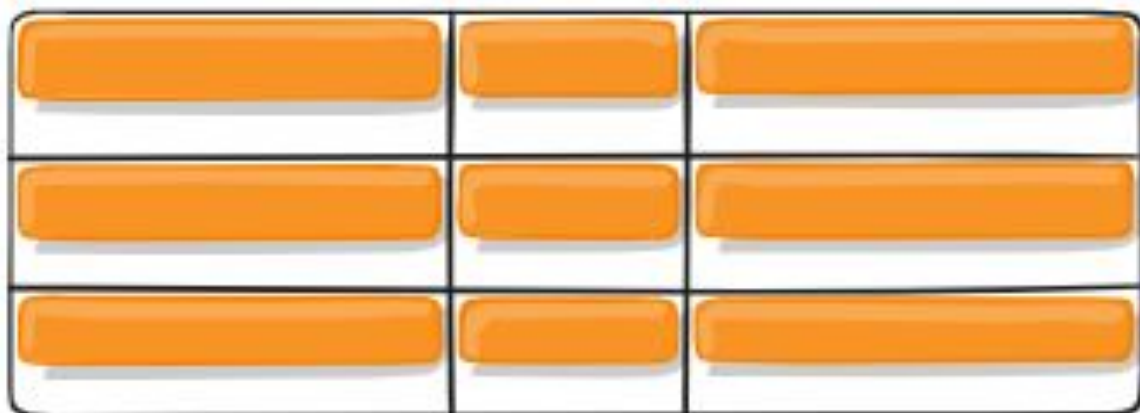
```
.grid-container {  
  display: grid;  
  align-items: center;  
}
```



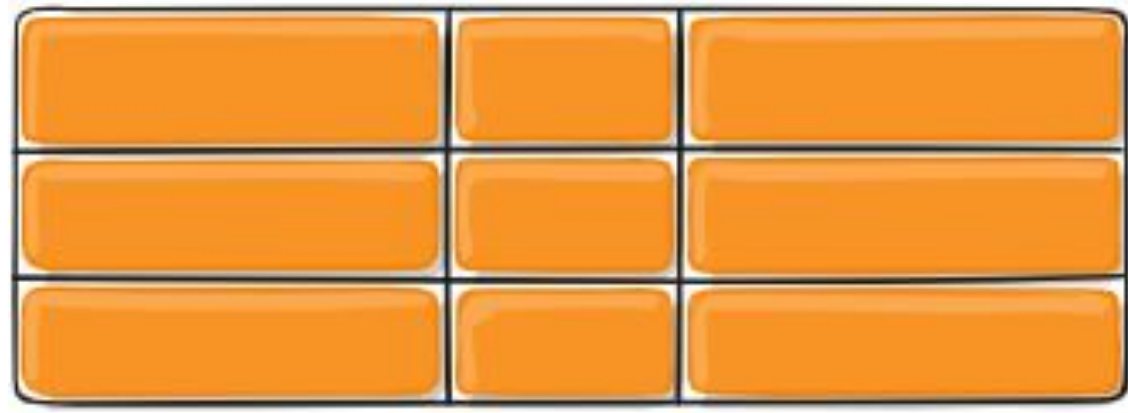
CSS Grid: principais conceitos

❖ align-item:

```
.grid-container {  
  display: grid;  
  align-items: start;  
}
```



```
.grid-container {  
  display: grid;  
  align-items: stretch;  
}
```



CSS Grid: principais conceitos

❖ align-item:

Sobre o *baseline*, alinha os itens com base na linha onde o texto “senta”.

- ❖ Somente faz diferença em elementos com muito texto e linhas, onde alinha o texto interno do elemento.

```
.grid-container {  
  display: grid;  
  align-items: baseline;  
}
```

CSS Grid: principais conceitos

❖ justify-content:

Alinha o grid inteiro dentro do container, e não somente os seus itens individualmente.

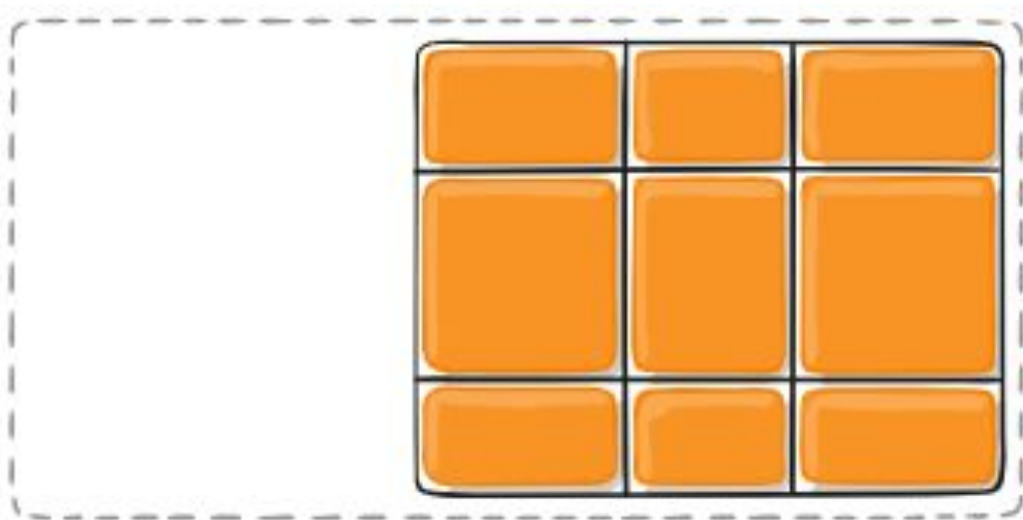
❖ Alinha o grid horizontalmente.

```
.grid-container {  
  display: grid;  
  justify-content: center;  
  /* start | end | center | stretch | space-around | space-between | space-evenly; */  
}
```

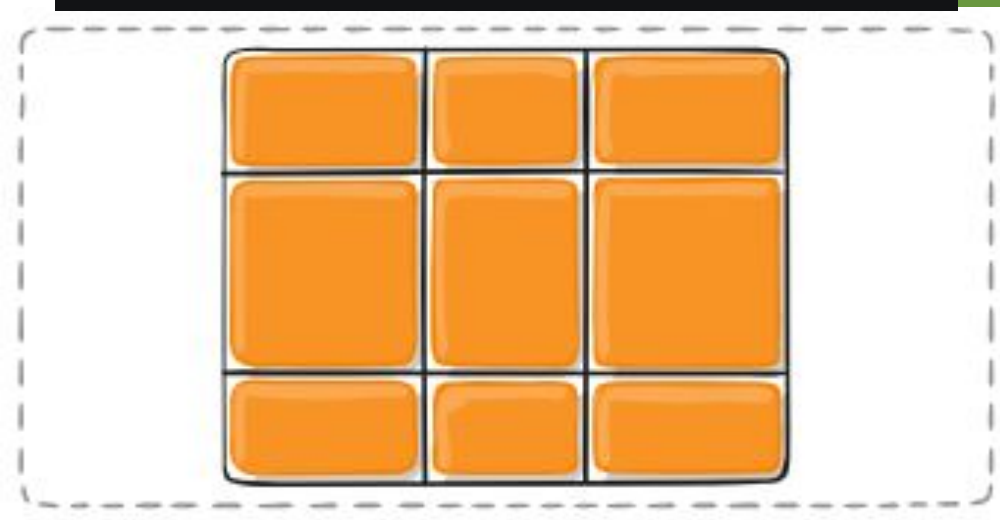
CSS Grid: principais conceitos

❖ justify-content:

```
.grid-container {  
  display: grid;  
  justify-content: end;  
}
```



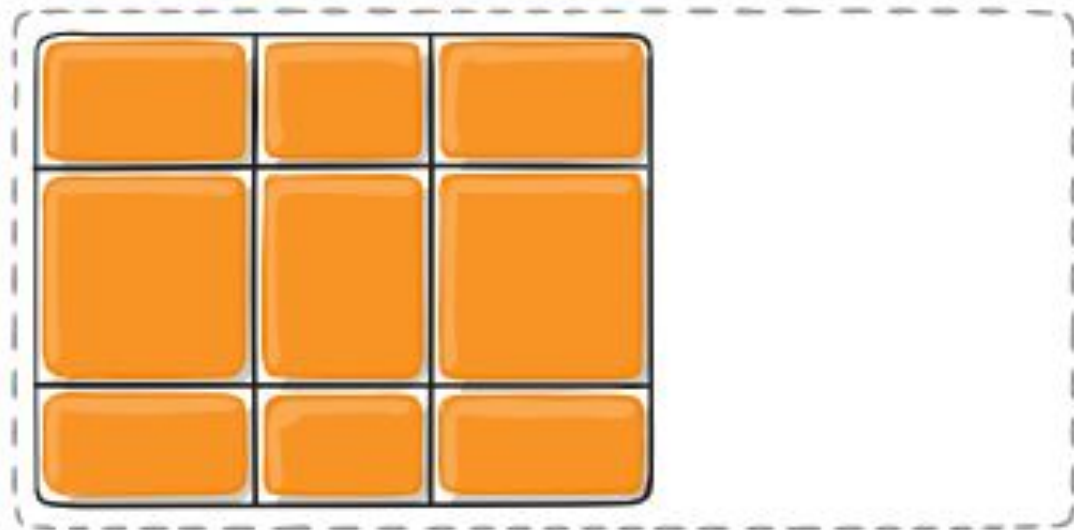
```
.grid-container {  
  display: grid;  
  justify-content: center;  
}
```



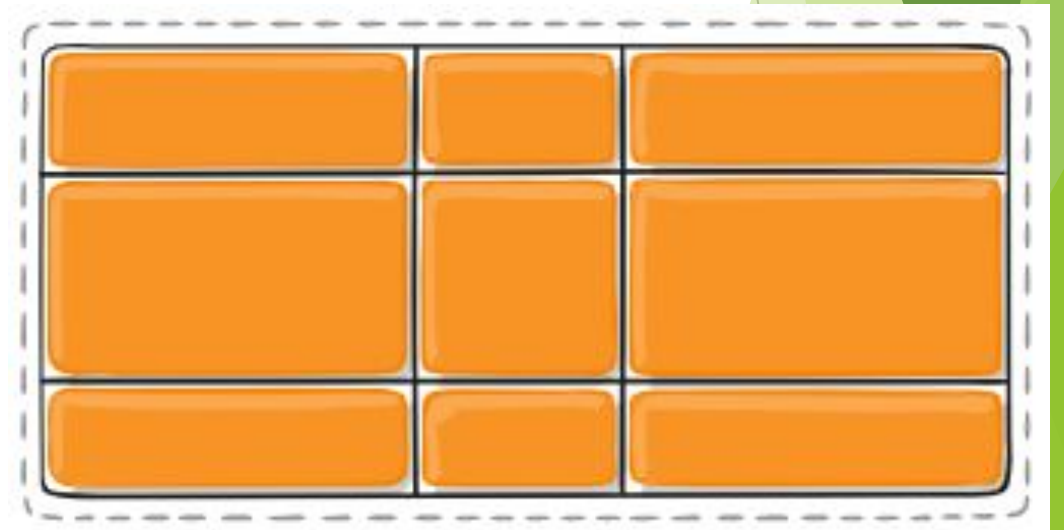
CSS Grid: principais conceitos

❖ justify-content:

```
.grid-container {  
  display: grid;  
  justify-content: start;  
}
```



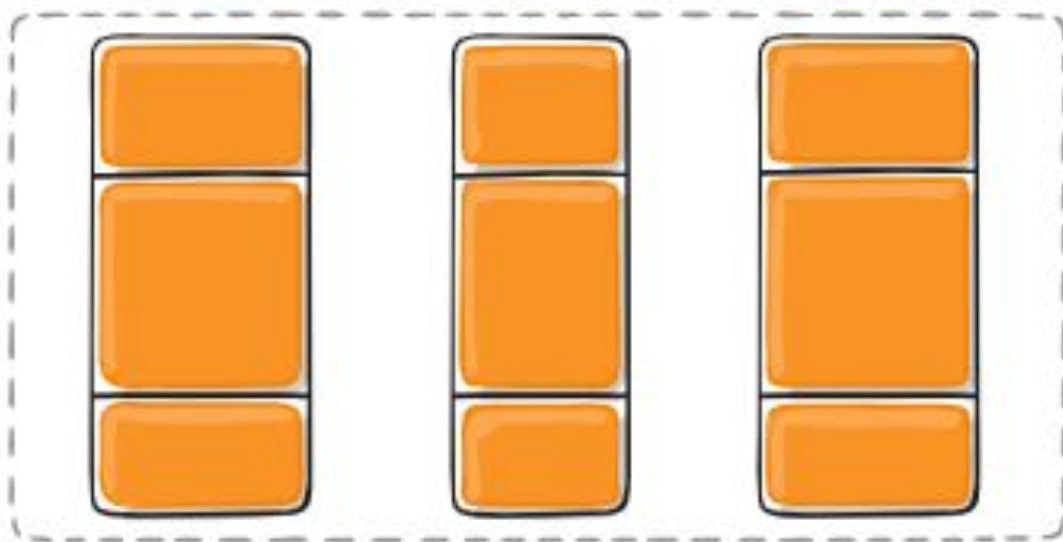
```
.grid-container {  
  display: grid;  
  justify-content: stretch;  
}
```



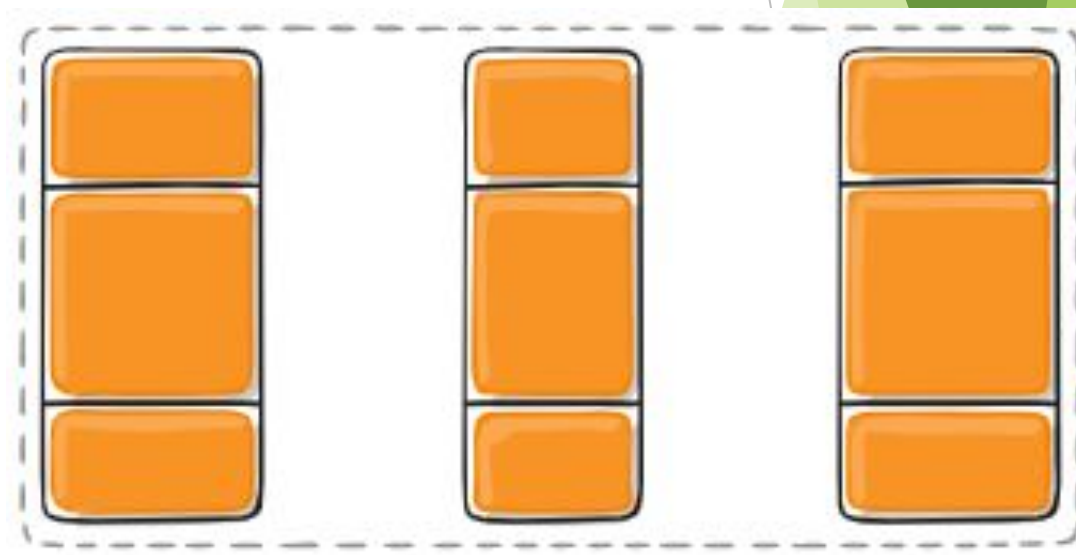
CSS Grid: principais conceitos

❖ justify-content:

```
.grid-container {  
  display: grid;  
  justify-content: space-around;  
}
```



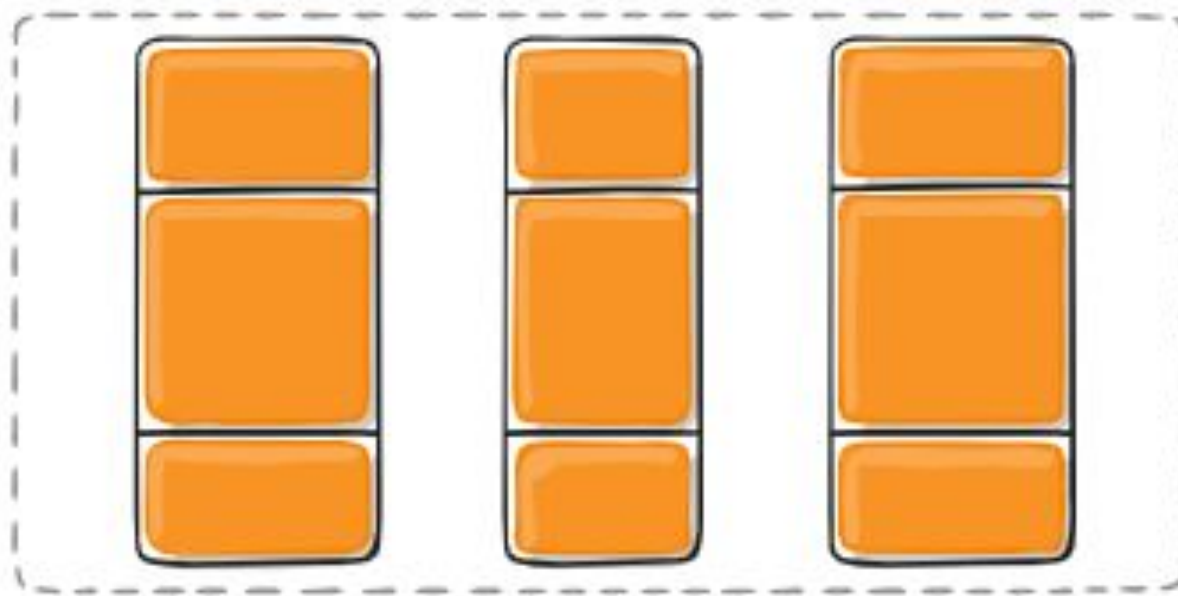
```
.grid-container {  
  display: grid;  
  justify-content: space-between;  
}
```



CSS Grid: principais conceitos

❖ justify-content:

```
.grid-container {  
  display: grid;  
  justify-content: space-evenly;  
}
```



CSS Grid: principais conceitos

❖ align-content:

Quando o tamanho dos itens do grid é menor (geralmente tamanho fixo) que o tamanho do **container do grid**, sobram espaços para alinhar o seu conteúdo dentro do container.

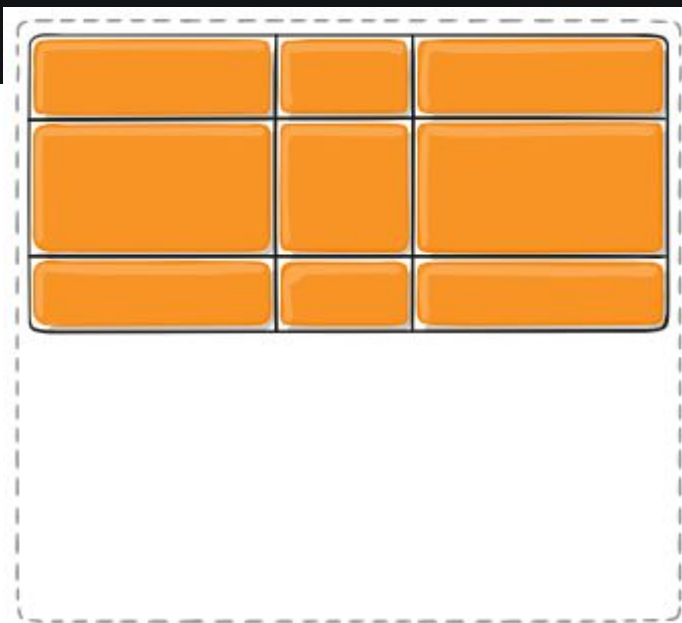
❖ Alinha o grid no eixo block, coluna.

```
.grid-container {  
  display: grid;  
  align-content: center;  
  /* start | end | center | stretch | space-around | space-between | space-evenly */  
}
```

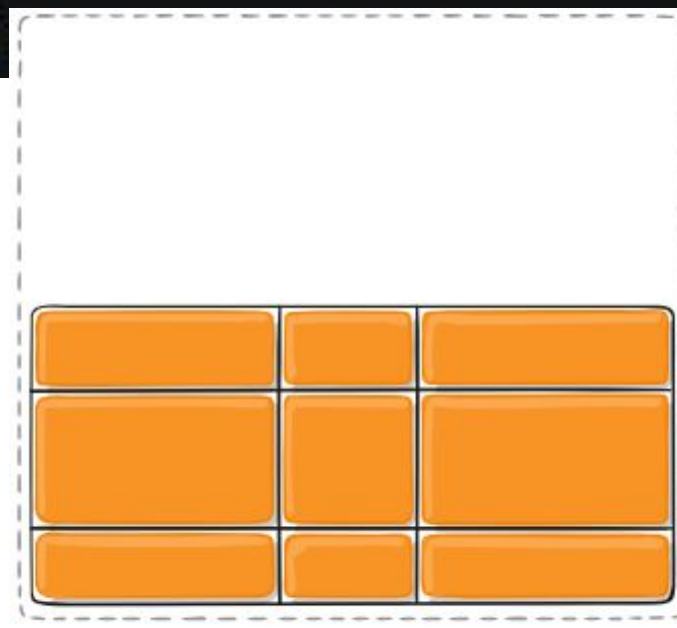
CSS Grid: principais conceitos

❖ align-content:

```
.grid-container {  
  display: grid;  
  align-content: start;  
}
```



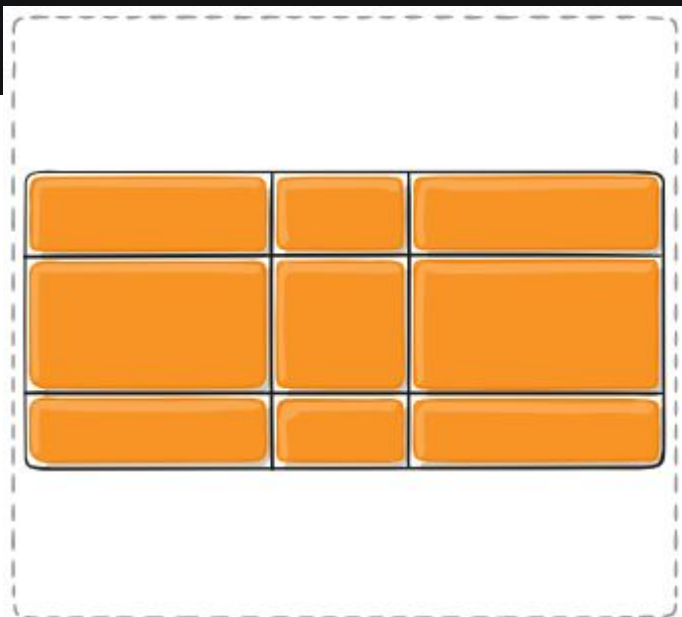
```
.grid-container {  
  display: grid;  
  align-content: end;  
}
```



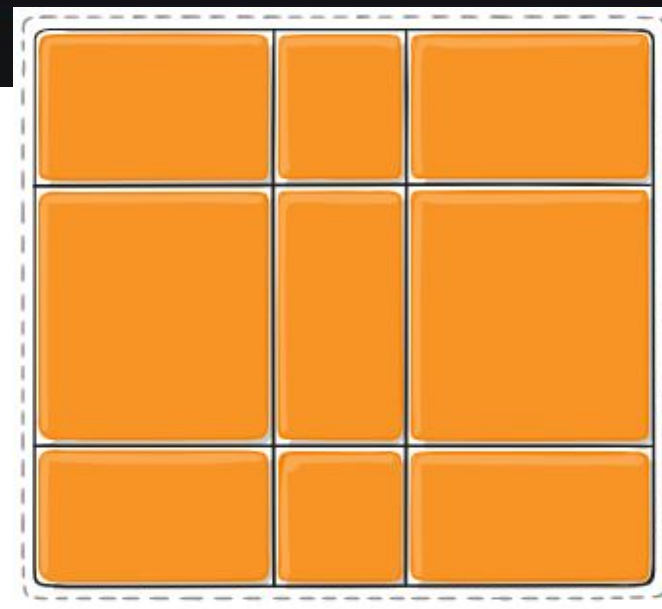
CSS Grid: principais conceitos

❖ align-content:

```
.grid-container {  
  display: grid;  
  align-content: center;  
}
```



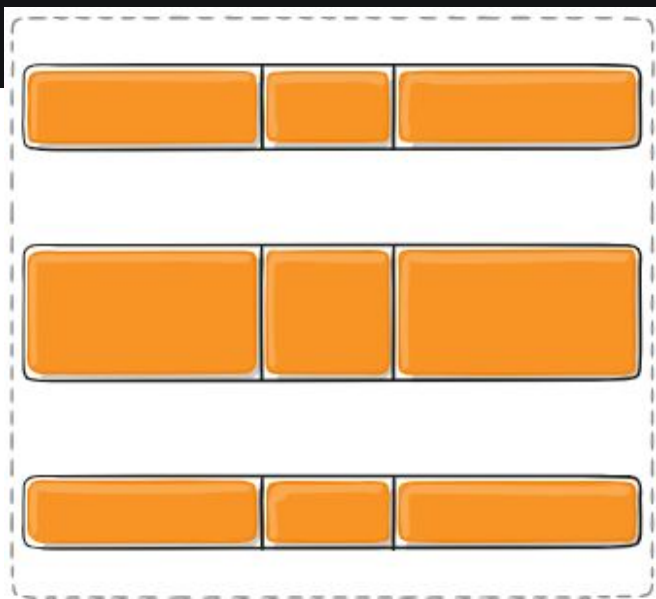
```
.grid-container {  
  display: grid;  
  align-content: stretch;  
}
```



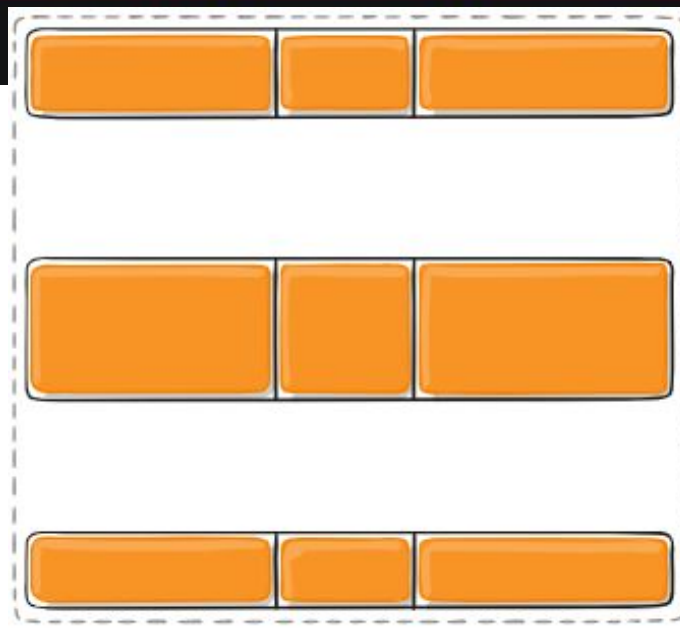
CSS Grid: principais conceitos

❖ align-content:

```
.grid-container {  
  display: grid;  
  align-content: space-around;  
}
```



```
.grid-container {  
  display: grid;  
  align-content: space-between;  
}
```



CSS Grid: principais conceitos

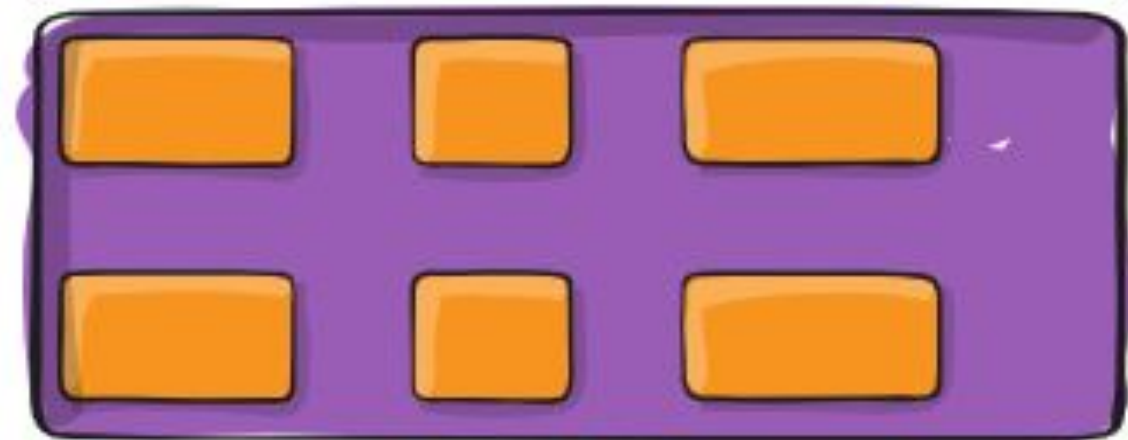
- ❖ **gap:** controla o espaço entre os itens de um flex container. Ela aplica esse espaçamento apenas entre os itens, não nas bordas externas.
- ❖ É uma forma abreviada que combina as propriedades *row-gap* e *column-gap*.

```
.grid-container {  
  display: grid;  
  gap: 10px 30px; /* row-gap column-gap */  
}
```

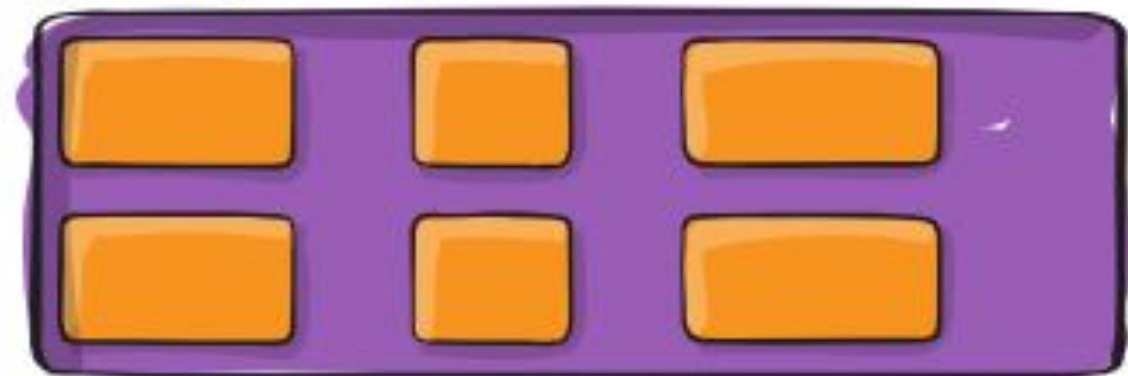
gap: 10px



gap: 30px



gap: 10px 30px



CSS Grid: principais conceitos

- ❖ **fr**: define a distribuição do espaço restante no container grid.

Divide o espaço disponível em 4 partes:

- ❖ 1fr = 1 parte (25%)
- ❖ 3fr = 3 partes (75%)

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
}
```



Fonte: autora, 2026

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect. The shapes are layered, with some appearing more prominent than others, and they extend towards the corners of the frame.

Dúvidas?

Exercício

CSS: exercício 3

- ❖ all 1: oculte o cabeçalho e rodapé principal de todas as páginas ao imprimir a página, criando uma regra específica de **media query** para isso (display none e media print).

```
/* all 1 */  
@media print {  
  .header,  
  .footer {  
    display: none;  
  }  
}
```

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

CSS: exercício 3

- ❖ **ct-tvs 1:** refatore os estilos CSS da página de categoria de produtos tvs de flex para **grid**.
- ❖ Defina o container grid e os itens filhos grid.

```
<> category_acessorios.html
<> category_tvs.html
<> products.html
# styles.css
```

```
/* ct-tvs 1 */
.products-list,
.products-list li {
  display: grid;
}

/* ct-tvs 1 */
/* grid container */
.products-list {
  padding: 20px;
}

/* grid item */
.product {
  border: 1px solid #ccc;
  padding: 10px;
  text-align: center;
}
```

CSS: exercício 3

- ❖ ct-tvs 2: as linhas devem ter espaçamento de 10px e as colunas de 20px (gap).
- ❖ O template deve ser em linha, itens lado a lado (grid-auto-flow), com 3 colunas (grid-template-columns) e 1 linha (grid-template-rows).

```
/* ct-tvs 2 */  
/* grid container */  
.products-list {  
  padding: 20px;  
  gap: 10px 20px; /* gap linha coluna */  
  grid-auto-flow: row; /* itens lado a lado */  
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas de largura igual */  
  grid-template-rows: 1fr; /* 1 linha de altura igual */  
}
```

<> category_acessories.html
<> category_tvs.html
<> products.html
styles.css

CSS: exercício 3

- ❖ ct-tvs 3: centralize os itens do container grid horizontalmente (eixo linha, justify-items).
- ❖ Aumente a altura dos itens para preencher o espaço vazio verticalmente (eixo coluna, align-items)
- ❖ Centralize o container grid verticalmente (eixo coluna, align-content)

```
.products-list {  
  padding: 20px;  
  gap: 10px 20px; /* gap linha coluna */  
  grid-auto-flow: row; /* itens lado a lado */  
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas de largura igual */  
  grid-template-rows: 1fr; /* 1 linha de altura igual */  
  justify-items: center; /* centraliza os itens no eixo da linha (direita-esquerda) */  
  align-items: stretch; /* aumente os itens para preencher o espaço disponível no eixo da coluna (cima-baixo) */  
  align-content: center; /* centraliza o grid no eixo da coluna (cima-baixo) */  
}
```

`<> category_acessories.html`
`<> category_tvs.html`
`<> products.html`
`# styles.css`

CSS: exercício 3

- ❖ **ct-tvs 4:** teste definir as linhas e colunas do grid criando áreas de template (grid-template-area).

```
/* grid container */
.products-list {
  padding: 20px;
  gap: 10px 20px; /* gap linha coluna */
  grid-auto-flow: row; /* itens lado a lado */
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas de largura igual */
  justify-items: center; /* centraliza os itens no eixo da linha (coluna) */
  align-items: stretch; /* aumenta os itens para preencher o espaço da linha */
  align-content: center; /* centraliza o grid no eixo da coluna (linha) */
  overflow: hidden;
  grid-template-areas: "product3 . product1" ". product2 .";
}
```

```
.product-1 {
  grid-area: product1;
}
```

```
.product-2 {
  grid-area: product2;
}
```

```
.product-3 {
  grid-area: product3;
}
```

```
<ul class="products-list">
  <li class="product product-1">...
</li>
  <li class="product product-2">...
</li>
  <li class="product product-3">...
</li>
</ul>
```

CSS: exercício 3

- ❖ **ct-tvs 5:** otimize para o celular a apresentação dos produtos (grid-template-columns), 3 colunas para telas grandes, 2 colunas para tablet e 1 coluna para celular.
- ❖ Oculte elementos para evitar quebra da listagem de produtos (overflow).

```
.products-list {  
  padding: 20px;  
  gap: 10px 20px; /* gap linha coluna */  
  grid-auto-flow: row; /* itens lado a lado */  
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas de largura igual */  
  justify-items: center; /* centraliza os itens no eixo da linha (direita-esquerda) */  
  align-items: stretch; /* aumente os itens para preencher o espaço disponível no eixo da coluna */  
  align-content: center; /* centraliza o grid no eixo da coluna (cima-baixo) */  
  overflow: hidden;  
}
```


CSS: exercício 3

- ❖ **ct-tvs 6:** crie as medias queries para celular (até 768px) e tablets (até 1024px).

```
/* ct-tvs 6 */  
/* tablet */  
@media (max-width: 1024px) {  
  .products-list {  
    grid-template-columns: 1fr 1fr;  
  }  
}  
  
/* celular */  
@media (max-width: 768px) {  
  .products-list {  
    grid-template-columns: 1fr;  
  }  
}
```

Salve esse código no seu email,
Google Drive ou GitHub!

Vídeo



Vídeo



Referência da aula

- ❖ COYIER, Chris. A complete guide to CSS Grid. CSS-Tricks, 2023. Disponível em: <https://css-tricks.com/complete-guide-css-grid-layout/>. Acesso em: 30 mar. 2026.
- ❖ W3SCHOOLS. CSS Grid Layout. W3Schools, [s.d.]. Disponível em: https://www.w3schools.com/css/css_grid.asp. Acesso em: 30 mar. 2026.
- ❖ THE NEXT WEB RESEARCH LAB. CSS layout comparison: float vs flex vs grid. The Next Web Research Lab, [s.d.]. Disponível em: <https://the-next-web-research-lab.github.io/en/fe-dev/javascript/float-flex-grid>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ ORIGAMID. CSS Grid Layout: guia completo. Origamid, [s.d.]. Disponível em:
<https://www.origamid.com/projetos/css-grid-layout-guia-completo/>. Acesso em: 30 mar. 2026.



univille.br